

OPERATIONS GUIDE

Future Deployments on the Live Server

A quick-reference runbook for pushing code changes to the running 7alpsglobal.com server — no re-setup, just the update loop, PM2/nginx cheat sheets, rollback, and troubleshooting.

Applies to the server already deployed and running at 7alpsglobal.com · repo: github.com/Eagleclaw123/7alps · Generated 2026-09-10

1. The standard update loop

Every future code change follows the same four moves: pull, rebuild whichever side changed, restart/reload. Run these on the server, inside the repo.

1 Pull the latest code

```
cd /var/www/7alps
git status           # should be clean – nothing local should ever be edited on the server
git pull origin main
```

2 Backend changed? Reinstall + restart

```
cd /var/www/7alps/backend
npm ci --omit=dev      # only needed if package.json/package-lock.json changed
pm2 restart 7alps-backend
pm2 logs 7alps-backend --lines 30 --nostream  # confirm it came back up clean
```

3 Frontend changed? Reinstall + rebuild

```
cd /var/www/7alps/frontend
npm ci                # only needed if package.json/package-lock.json changed
npm run build          # overwrites frontend/dist – nginx serves this directly, no restart
                        # needed
```

nginx serves `dist/` as static files, so a rebuild takes effect immediately on the next page load — no nginx reload required. If the site still shows old content after a rebuild, it's almost always a **browser cache** issue — hard refresh (Cmd/Ctrl+Shift+R) before assuming the deploy failed.

4 nginx config itself changed?

Only needed if you edited `deploy/nginx.conf` in the repo (new route, new proxy rule, etc.) — most day-to-day deploys never touch this.

```
sudo cp /var/www/7alps/deploy/nginx.conf /etc/nginx/sites-available/7alpsglobal.com
sudo nginx -t                # always test before reloading
sudo systemctl reload nginx
```

Certbot has already edited this file once (added the 443/SSL block and the HTTP→HTTPS redirect). Copying `deploy/nginx.conf` from the repo over the live file will **wipe those SSL directives** unless you've also updated the repo copy to match. Safer default: hand-edit `/etc/nginx/sites-available/7alpsglobal.com` directly on the server for nginx-only changes, and only copy from the repo if you deliberately re-synced the repo's copy with the live SSL config first.

5 Environment variable changed?

Edit the real file directly on the server — it is not in git (gitignored on purpose, since it holds secrets):

```
nano /var/www/7alps/backend/.env          # backend secrets/config
pm2 restart 7alps-backend                 # env vars only load at process start
```

For a **frontend** env change (e.g. rotating the Google Maps key), edit `frontend/.env.production` and rebuild — Vite bakes these into the JS bundle at build time, so a restart alone does nothing:

```
nano /var/www/7alps/frontend/.env.production
cd /var/www/7alps/frontend && npm run build
```

2. What to run, by change type

You changed...	What to run
Backend route/controller/model code	<code>pm2 restart 7alps-backend</code>
Backend <code>package.json</code> (new dependency)	<code>npm ci --omit=dev</code> then restart
Backend <code>.env</code>	edit on server directly, then restart
Frontend React/component code	<code>npm run build</code> (no restart needed)
Frontend <code>package.json</code> (new dependency)	<code>npm ci</code> then <code>npm run build</code>
Frontend <code>.env.production</code>	edit on server, then <code>npm run build</code> (env is baked at build time)
<code>deploy/nginx.conf</code> in the repo	copy over live config, <code>nginx -t</code> , reload — see the SSL warning above
Nothing, just want to see current state	<code>pm2 status</code> , <code>sudo systemctl status nginx</code>

3. PM2 cheat sheet

```
pm2 status          # is it up? memory/CPU? restart count?
pm2 logs 7alps-backend # live tail (Ctrl+C to exit)
pm2 logs 7alps-backend --lines 100 --nostream # last 100 lines, no follow
pm2 restart 7alps-backend # hard restart (brief downtime, single fork instance)
pm2 reload 7alps-backend # graceful reload — no-op in fork mode with 1 instance, same
as restart here
pm2 stop 7alps-backend # stop without removing from PM2's process list
pm2 monit              # live CPU/memory dashboard
pm2 save               # persist current process list — run after any pm2
start/stop/delete
```

Log files also live on disk at `backend/logs/out.log` and `backend/logs/error.log` (configured in `ecosystem.config.js`), in case `pm2 logs` has rotated past what you need.

4. Rolling back a bad deploy

If a deploy breaks something and you need to get back to the last known-good commit fast:

```
cd /var/www/7alps
git log --oneline -10          # find the last good commit hash
git checkout <good-commit-hash> # detached HEAD, that's fine for a quick rollback

cd backend && npm ci --omit=dev && pm2 restart 7alps-backend
cd ../frontend && npm ci && npm run build
```

Once the underlying issue is actually fixed, return to the branch tip properly:

```
cd /var/www/7alps
git checkout main
git pull origin main
# then repeat the normal backend/frontend rebuild steps above
```

5. SSL certificate renewal

Renewal is automatic — certbot installs a systemd timer that runs twice a day and only actually renews when the cert is within 30 days of expiry. You generally don't need to do anything. To check on it:

```
sudo systemctl status certbot.timer    # confirm the timer is active
sudo certbot renew --dry-run           # simulate a renewal without actually replacing the cert
sudo certbot certificates               # see current cert, issuer, and expiry date
```

6. Troubleshooting quick reference

Symptom	Likely cause / first check
Site shows 502 Bad Gateway	Backend process is down or crashed. <code>pm2 status</code> , then <code>pm2 logs 7alps-backend</code> for the crash reason.
Site loads but shows old content	Either you forgot to run <code>npm run build</code> on the frontend, or it's just your browser cache — hard refresh first.
CORS error in browser console	Only matters if something calls the API from a different origin than <code>7alpsglobal.com</code> . Check <code>CORS_ORIGINS</code> in <code>backend/.env</code> .
Database connection errors in <code>pm2 logs</code>	Check MongoDB Atlas → Network Access — if the server's IP ever changes (e.g. a new ephemeral IP after a reboot on some cloud providers), it needs to be re-added to the allowlist.
Login/session issues after a deploy	Cookies are <code>httpOnly</code> + <code>secure</code> + <code>sameSite=none</code> in production — this requires HTTPS to actually be working. Confirm the SSL cert is valid, not just that nginx is up.
Emails (OTP, notifications) stop sending	Check <code>RESEND_API_KEY</code> is still set and the sending domain is still verified in the Resend dashboard.
Payments fail at checkout	Check <code>RAZORPAY_KEY_ID</code> / <code>RAZORPAY_KEY_SECRET</code> are still valid — the backend returns a graceful 503 if they're ever blanked out.

Never run `npm run seed` on this server without reading the script first. It has wiped the live Products collection once before during development. There should be no reason to run any `seed` / `seed:b2b` / `seed:categories` script against production data — those are dev-only bootstrap scripts.

Zero-downtime deploys. The backend currently runs as a single PM2 fork-mode process, so `pm2 restart` causes a brief (typically sub-second) gap in availability while it restarts. If that ever becomes a problem, PM2 cluster mode with 2+ instances and `pm2 reload` (instead of `restart`) would give truly zero-downtime deploys — not set up today since traffic doesn't warrant it yet.